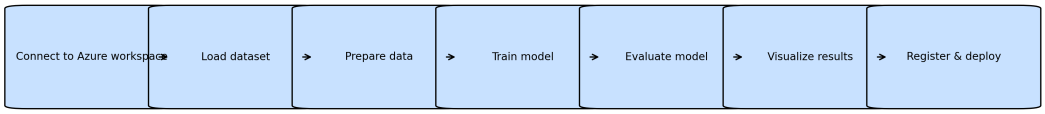
Anomaly Detection in Azure ML Studio

This notebook demonstrates a proof of concept fraud detection pipeline built in Azure Machine Learning. It shows how our team connects to the Azure cloud, retrieves an existing credit card transaction dataset, prepares the data, trains an anomaly detection model, evaluates its performance, and prepares it for deployment. The goal is to explore whether unsupervised anomaly detection can help our financial services company identify unusual transactions that might indicate credit card fraud.

Below is a simplified diagram of the process we follow from start to finish. Each box represents a step in the pipeline and the arrows show the flow of data and decisions.



To help you map the Azure ML components to their purpose, the following table lists the main resources involved and what role each one plays.

Azure ML Component	Role
Workspace	The cloud environment that holds our experiments, data assets, compute targets and models.
Dataset	A registered copy of the credit card transaction data that we load from Azure for analysis.
Compute	The virtual machines or clusters that run the Python code to prepare data and train the model.
Environment	A snap shot of the software packages and versions used to ensure the code runs consistently.
Experiment	A grouping of related runs that tracks metrics, outputs and logs for each training session.
Model	The trained Isolation Forest that we can save and reuse once it performs well.
Registry/Deployment	Where we store the trained model so it can be exposed as an end point or integrated into applications.

In the following sections we describe each step of the process in plain language, focusing on what it means rather than how the code works.

In [ ]:
```
# Step 1: Import Packages and Connect to your Azure Workspace
from azureml.core import Workspace, Dataset         # see https://pypi.org/p
```

```
import pandas as pd                                    # see https://pandas.pyc
from sklearn.ensemble import IsolationForest           # see https://scikit-lea
from sklearn.metrics import classification_report      # see https://scikit-lea
from azureml.core.model import Model                   # see https://docs.micro
```

Step 2: Load the credit card transaction data

Next we load the registered credit card transaction data from our Azure workspace. We convert it into a pandas table so we can explore and manipulate it in Python. Please note that this dataset is large; if you run this notebook locally it may take a few minutes to download because it contains many transaction records.

In [ ]:
```
# You only need to run this if you've imported this notebook to Azure AI Mac
# in which case you'll also need to upload the config.json file to the same
# and then execute this code to determine the current working directory.
import os
print("Current working directory:", os.getcwd())
print("Files in this directory:", os.listdir())
```

In [ ]:
```

```

In [ ]:
```
# if you're running locally then use this ...
path = None

# alternatively, if you're running in Azure AI Machine Learning Studio - Not
# (make sure to upload the config.json file to the same directory as this no
#  and then execute this code to determine the current working directory.
path='Users/[REPLACE-THIS-WITH-YOUR-USERNAME]/config.json'
ws = Workspace.from_config(path=path)
dataset = Dataset.get_by_name(ws, name='creditcard_fraud')
df = dataset.to_pandas_dataframe()
df.head()
```

Step 3: Prepare the data

We normalise the transaction amount column so that the values are on a similar scale to other features. We then create two tables: `X` contains the input features that the model will look at, and `y` contains the labels showing which transactions are fraudulent. We drop the `Time` column because it does not help the model spot anomalies. Normalising and splitting the data helps the model learn effectively.

In [ ]:
```
df['Amount'] = (df['Amount'] - df['Amount'].mean()) / df['Amount'].std()
X = df.drop(columns=['Class', 'Time'])
y = df['Class']
```

Step 4: Train the model

We use an Isolation Forest to learn patterns from normal transactions and identify unusual ones. This algorithm does not need labelled examples of fraud; instead it

isolates points that look different from the majority. We set the `contamination` parameter based on the known fraction of fraud cases in the data so the model knows roughly how many anomalies to expect.

In [ ]:
```
model = IsolationForest(contamination=0.0017, random_state=42)
model.fit(X)
y_pred = model.predict(X)
y_pred = [1 if x == -1 else 0 for x in y_pred]
```

Step 5: Evaluate the model

After training we compare the model's predictions with the known labels. The table of metrics shows how often the model correctly identifies normal and fraudulent transactions. The model is very good at recognising normal transactions but misses many fraudulent ones. For example, the precision for the fraud class is about 29 percent and the recall is about 28 percent. This means that when the model flags a transaction as fraud it is right about one third of the time and it only finds about one quarter of the actual fraud cases. Because fraud is rare, the overall accuracy is almost perfect but it hides the fact that many frauds are missed.

In [ ]:
```
# Step 5: Evaluate Model
print(classification_report(y, y_pred))
```

Step 6: Register the model (optional)

If we want to reuse this trained model later or deploy it to production we can save it as a file and register it in our Azure workspace. Registration stores the model along with its version so other services can access it. In a real system we would register the model after we are satisfied with its performance.

In [ ]:
```
import joblib                                          # see https://joblib.rea
                                                       #     Joblib is a set of

joblib.dump(model, 'isolation_forest.pkl')
Model.register(model_path='isolation_forest.pkl',
               model_name='creditcard_if_model',
               workspace=ws)
```

Step 7: Visualise predicted anomalies

We count how many transactions the model labels as normal and how many it labels as anomalies. Because fraud cases are rare we expect only a small number of anomalies. A bar chart provides a quick check on whether the number of predicted frauds is reasonable. If the model flags too many transactions we may end up with unnecessary investigations; if it flags too few we may miss fraud cases.

In [ ]:
```
import matplotlib.pyplot as plt
import seaborn as sns

# Add predictions to the original dataframe
df['predicted_anomaly'] = y_pred

# Count of predicted anomalies
sns.countplot(x='predicted_anomaly', data=df)
plt.title('Count of Predicted Anomalies')
plt.xlabel('Anomaly (1) vs Normal (0)')
plt.ylabel('Count')
plt.show()
```

Step 8: Compare transaction amounts by prediction

A boxplot compares the distribution of transaction amounts for transactions that the model marks as normal and those it marks as anomalies. This helps us see whether the model is mostly reacting to extreme amounts. If the amounts flagged as anomalies have a wider range it suggests the model is sensitive to very high or low transactions. This insight can guide feature engineering.

In [ ]:
```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df, x='predicted_anomaly', y='Amount')
plt.title('Transaction Amount by Prediction Class')
plt.show()
```

Step 9: Understand feature importance with SHAP

The SHAP beeswarm plot shows which features influenced the model's decisions. Each dot is one transaction; its colour indicates whether the feature value is high or low and its position along the horizontal axis shows whether it pushes the model towards predicting fraud or normal. This plot helps us identify which variables the model relies on and provides transparency for stakeholders.

In [ ]:
```
import shap

explainer = shap.Explainer(model, X)
shap_values = explainer(X[:100])
shap.plots.beeswarm(shap_values)
```

Business Impact Assessment

Detecting fraud involves a delicate balance between catching fraudulent transactions and keeping customer experiences smooth. Errors can occur in two ways: **false positives** (flagging a legitimate transaction as fraud) and **false negatives** (missing an actual fraud).

Cost and benefit of false positives and false negatives

Research on fraud detection shows that false positives create customer friction and lost revenue, while false negatives lead to direct financial losses and reputational damage. A high rate of false positives means legitimate customers are inconvenienced, leading to abandoned purchases and extra investigation costs. False negatives are more dangerous because successful fraud results in chargebacks, stolen goods and potential regulatory penalties. Our current isolation forest model has low recall on fraud cases (around 28 percent), which means we are likely to miss many fraudulent transactions. Improving recall will reduce false negatives but may increase false positives. Stakeholders need to decide where to set the threshold based on the acceptable trade off between customer experience and fraud losses.

Recommendations for model improvement

- **Use supervised models**: Supervised algorithms like random forests, gradient boosting or neural networks trained on labelled fraud data can achieve higher accuracy and reduce errors. Combining supervised and unsupervised methods may capture both known and novel fraud patterns.
- **Adjust thresholds and contamination**: Tuning the contamination parameter and decision threshold can balance the rate of false positives and false negatives. Organisations should experiment with different settings and choose the one that aligns with their risk appetite.
- **Address class imbalance**: Fraud datasets are highly imbalanced. Techniques like resampling or synthetic sampling can help the model pay more attention to rare fraud cases.
- **Feature engineering**: Incorporating additional features such as geographic distance between transactions, merchant categories and customer behaviour patterns can improve predictive power.
- **Continuous learning**: Machine learning models should be retrained regularly with new transaction data. Adaptive learning allows the system to learn new fraud patterns as attackers change their tactics.

Risk assessment and mitigation

- **Data privacy and compliance**: Ensure the model complies with data protection laws. Only use anonymised or tokenised transaction data and follow regulatory guidance for storing and processing personal information.
- **Monitoring and alerting**: Deploy monitoring to track model performance and drift. If the false positive rate increases or recall drops, retrain or adjust the model. Human analysts should review flagged transactions, especially in ambiguous cases.
- **Ethical considerations**: Avoid biases by validating that the model does not unfairly impact certain groups. Use SHAP or similar tools to explain predictions and detect

bias. Maintain an ethical review process and obtain legal counsel where necessary.

Stakeholder communication plan

Transparency builds trust. The OECD principles for AI transparency highlight that organisations should foster a general understanding of AI systems, make users aware when they interact with AI, and enable affected parties to understand and challenge outcomes. We propose the following communication plan:

- **Explain the purpose and scope**: Brief stakeholders on why the model exists, what data it uses and how it will be used in the business process.
- **Share performance metrics**: Regularly report precision, recall, false positive rate and false negative rate so decision makers understand the trade offs.
- **Document decisions and changes**: Keep records of training data sources, model versions and parameter settings. Provide accessible documentation and visualisations to non technical stakeholders.
- **Engage stakeholders**: Hold periodic workshops or briefings to gather feedback from customer service teams, risk managers and executives. Tailor the level of detail to each audience; for example, technical teams may need detailed reports, while executives may prefer summaries and charts.
- **Provide avenues for appeal**: Ensure there is a process for customers or internal users to challenge decisions made by the model. Feedback can be used to improve the model and address unintended consequences.

By following these recommendations and maintaining clear communication, we can make informed decisions about deploying the anomaly detection model and continually improve its effectiveness.